C++

Manipulator

`endl` Insert newline and flush stream

`setw(int w)` Set field width to w. It only applies to the next value to be inserted, then it is reset to default (0)

`setfill(int c)` Set the fill character to c. Default is space.

`left` Append fill characters at the end

`right` Insert fill characters at the beginning

`setprecision(int d)` Set the floating point precision to d.

`fixed` Floating-point values are written using fixed-point notation: the value is represented with exactly as many digits in the decimal part as specified by the

Macros

`#define` Function like macros (C also supports this)

```cpp
#include <iostream>

#define SQUARE(x) (x) * (x)

int sqr(int x) {
    return x * x;
}

int main() {
    std::cout << SQUARE(3) << std::endl;
    std::cout << SQUARE(3.1 + 1) << std::endl << std::endl;
    std::cout << sqr(3) << std::endl;
    std::cout << sqr(3.1 + 1) << std::endl;
    return 0;
}
```

Inline

- **Benefit of Macros (~Drawback of function)**
  - No function call needed, hence no overhead (like required for function call)
- **Drawbacks of Macros (~Benefits of function)**
  - Sometimes you get results you might not have expected
  - No type checking

When a function is defined as inline, it is just a request (NO GUARANTEE) to the compiler to expand it in line when it is called.

- It is as if the whole code of the inline function gets inserted or substituted at the point of the inline function call.

- The compiler is intelligent - Same variable names in the calling function and inline function will not clash.

Inline functions must be defined before they are called so that the compiler knows the exact code.

- Inline functions are not the same as macros – macros are handled by the preprocessor and inline functions are handled by the compiler.

Inline functions CANNOT:

- Be recursive in nature
- Contain static variables

Default arguments

```
int sum(int n1, int n2, int n3 = 111, int n4 = 222);

int sum(int n1, int n2, int n3 = 4, int n4); // Invalid
```

Function overloading

Signature of the function includes number, type, and order of parameters

I. Exact match
II. Promotions

- Integral promotions: bool, char, short, enum to int
- Floating-point promotions: float to double

III. Standard type conversions

- e.g. double to float, int to long, int to float, bool to short, bool to float, etc.
  IV. User-defined conversions (related to classes, ignore for now)

Struct and Classes

C++ structures are nothing but classes. The only difference is that, by default, members of a class are private while, by default, members of a structure are public (for compatibility with the C language).

Class is a way to bind data and associated functions together

Methods defined within the class definition become inline by default. Hence, only small functions should be defined within the class definition.

Methods defined outside the class need to be fully qualified with the class name and scope resolution operator. They are not inlined by default but can be declared inline explicitly using the `inline` keyword.

`this` keyword

The `this` keyword is like a constant pointer to the object which invoked the method.

`static` keyword

- Only one copy is created for a static data member.
- All objects of that class share the same copy.
- It is stored in the data section.
- Even when the class does not have any objects, static data members can be accessed and used.

```
class GfG {
public:
    static int i;
    GfG(){
        // Do nothing
    };
};
```

```
int GfG::i = 1;
```

Static data members must be defined outside the class (without the `static` keyword). The name of the class and the scope resolution operator is used.

// static method can access only static members of the class

Friend Function

Friend functions:

- We can allow outside functions to access private members of a class.
- To make an outside function friendly, we need to declare it as a friend within the class definition using the `friend` keyword.

```
class base {
    private:
        int private_variable;
    protected:
        int protected_variable;
    public:
        base() {
            private_variable = 10;
            protected_variable = 99;
        }
        friend void friendFunction(base& obj);
};

void friendFunction(base& obj) {
    cout << "Private Variable: " << obj.private_variable << e
ndl;
    cout << "Protected Variable: " << obj.protected_variable;
}
```

Member functions of other classes can also be declared as friend functions:

```
class Y {
    friend int X::fun();
};
```

Const Member Functions

- If a member function does not alter any data member of the object, then it may be declared as a `const` method.

Benefits of Const Member Functions

- The developer will not modify the state of the object from `const` member functions by mistake.
- A `const` object can call `const` member functions, but it cannot call other non-const member functions.

argument discards qualifiers
const obj can't call non-const method

Constructors and Destructors in C++

Constructors

> In C++, objects can be initialized using constructors. A constructor is a special member function of a class that is used to initialize objects of its class type. The name of the constructor is the same as the class name.

Default Constructor

- When no constructor has been defined for a class, the compiler supplies a default constructor

- The default constructor provided by the compiler is a "do-nothing" kind
- As soon as a constructor is defined for a class (with or without arguments), the compiler no longer provides a default constructor

Copy Constructor

- A copy constructor of class T has the first parameter as `T&`, `const T&`, `volatile T&`, or `const volatile T&`
- Either there are no other parameters, or the rest of the parameters all have default values
- All of our copy constructors will have `T&` or `const T&` as the first parameter
- Use `const T&`, unless you know that you need `T&`
- When no copy constructor has been defined for a class, the compiler supplies a default copy constructor
- The default copy constructor provided by the compiler does a byte-wise copy from one object to another object

⚠ **Important Note**: For the copy constructor of class T, we are passing a reference variable of type T as the first argument. We cannot pass the argument by value, because it will lead to an infinite loop.

Initializing const Members

- `const` members cannot be initialized inside the constructor body
- `const` members must be initialized before entering the constructor body

Example:

```
class Test {
    const int t;
    int i;
public:
    Test(int num): t(num), i(num * 2) {
        i = num * 3;
    }
```

```
    void print();
};
```

Destructors

> A destructor is a special member function that is called just before the lifetime of an object ends.

- An object's lifetime ends only after its destructor finishes its execution
- Execution of the destructor is the last thing in the lifetime of an object
- Auto and global objects are destroyed in reverse order of their creation

Memory Management in Destructors

- If we do not release dynamic memory allocated for object's data members, it will lead to memory leaks
- Once an object's lifetime ends, its destructor will be called
- After execution of the destructor, the object's memory would be freed
- If dynamic memory allocated for data members has not been freed already and if we do not free it in the destructor, it cannot be freed until the end of program execution
- This is because we will lose the pointer to dynamic memory allocated for the data member, as the pointer is stored in the data member of the object

Example:

```
class Str {
    int len;
    char *ptr;
public:
    Str(const char *ptr) {
        cout << "Constructor called" << endl;
        len = strlen(ptr);
```

```
        this->ptr = new char[len + 1];
        strcpy(this->ptr, ptr);
    }

    void print() {
        cout << ptr << endl;
    }

    ~Str() {
        cout << "Destructor called" << endl;
        delete ptr;  // Prevent memory leak
    }
};
```

Exception Handling

catch all

```
catch(...) { // Notice use of ellipsis
    cout << "catch all called!!\n";
}
```

Rethrowing Exceptions

- Catch blocks can rethrow the same exception using the `throw` keyword without an expression.
- When an inner catch block throws an exception, it bypasses remaining inner catch blocks and is matched directly against outer catch blocks.

Custom Exception Example

```
cpp
Copy
class DivideByZeroException: public std::exception {
public:
    virtual const char* what() const throw() {
        return "DivideByZeroException raised!\n";
    }
};
```

Exceptions in Constructors

- Exceptions from constructors should be handled within the constructor first.
- If needed, exceptions can be thrown outside the constructor if the object cannot be constructed successfully.
- Dynamically allocated memory should be deleted/freed in catch blocks within the constructor.
- Memory of the actual object (including other member objects) will be freed automatically.
- Before throwing an exception out of the constructor, the compiler emits code to:
    - Destroy already constructed members and base objects (by calling respective destructors)
    - Delete the memory of the object under construction without calling its destructor

Why Destructors Default to noexcept

- Destructors are called during stack unwinding to delete all auto objects on the stack from the point where an exception was thrown to where it's handled.
- If a destructor throws an exception during stack unwinding:
    - The original stack unwinding process will never complete.

- Stack unwinding for the new exception (thrown from the destructor) would start.
- This creates a messy situation where the first exception's stack unwinding is never completed.